# Using the Native SDK for Node.js

Version 2024.2
2024-10-08

InterSystems Corporation 1 Memorial Drive Cambridge MA 02142 www.intersystems.com

*Using the Native SDK for Node.js*
PDF generated on 2024-10-08
InterSystems IRIS® Version 2024.2
Copyright © 2024 InterSystems Corporation
All rights reserved.

InterSystems®, HealthShare Care Community®, HealthShare Unified Care Record®, IntegratedML®, InterSystems Caché®, InterSystems Ensemble®, InterSystems HealthShare®, InterSystems IRIS®, and TrakCare are registered trademarks of InterSystems Corporation. HealthShare® CMS Solution Pack™, HealthShare® Health Connect Cloud™, InterSystems® Data Fabric Studio™, InterSystems IRIS for Health™, InterSystems Supply Chain Orchestrator™, and InterSystems TotalView™ For Asset Management are trademarks of InterSystems Corporation. TrakCare is a registered trademark in Australia and the European Union.

All other brand or product names used herein are trademarks or registered trademarks of their respective companies or organizations.

This document contains trade secret and confidential information which is the property of InterSystems Corporation, One Memorial Drive, Cambridge, MA 02142, or its affiliates, and is furnished for the sole purpose of the operation and maintenance of the products of InterSystems Corporation. No part of this publication is to be used for any other purpose, and this publication is not to be reproduced, copied, disclosed, transmitted, stored in a retrieval system or translated into any human or computer language, in any form, by any means, in whole or in part, without the express prior written consent of InterSystems Corporation.

The copying, use and disposition of this document and the software programs described herein is prohibited except to the limited extent set forth in the standard software license agreement(s) of InterSystems Corporation covering such programs and related documentation. InterSystems Corporation makes no representations and warranties concerning such software programs other than those set forth in such standard software license agreement(s). In addition, the liability of InterSystems Corporation for any losses or damages relating to or arising out of the use of such software programs is limited in the manner set forth in such standard software license agreement(s).

THE FOREGOING IS A GENERAL SUMMARY OF THE RESTRICTIONS AND LIMITATIONS IMPOSED BY INTERSYSTEMS CORPORATION ON THE USE OF, AND LIABILITY ARISING FROM, ITS COMPUTER SOFTWARE. FOR COMPLETE INFORMATION REFERENCE SHOULD BE MADE TO THE STANDARD SOFTWARE LICENSE AGREEMENT(S) OF INTERSYSTEMS CORPORATION, COPIES OF WHICH WILL BE MADE AVAILABLE UPON REQUEST.

InterSystems Corporation disclaims responsibility for errors which may appear in this document, and it reserves the right, in its sole discretion and without notice, to make substitutions and modifications in the products and practices described in this document.

For Support questions about any InterSystems products, contact:

**InterSystems Worldwide Response Center (WRC)**
Tel: +1-617-621-0700
Tel: +44 (0) 844 854 2917
Email: support@InterSystems.com

# Table of Contents

**1 Introduction to the Native SDK for Node.js ..... 1**

**2 Calling Class Methods from Node.js ..... 3**
2.1 Calling Class Methods from Node.js ..... 3
2.2 Passing Arguments by Reference ..... 5

**3 Controlling Database Objects from Node.js ..... 7**
3.1 Introducing the Node.js External Server ..... 7
3.2 Creating Node.js Inverse Proxy Objects ..... 8
3.3 Controlling Database Objects with IRISObject ..... 9

**4 Accessing Global Arrays with Node.js ..... 11**
4.1 Introduction to Global Arrays ..... 11
4.1.1 Glossary of Global Array Terms ..... 13
4.1.2 Global Naming Rules ..... 14
4.2 Fundamental Node Operations ..... 14
4.3 Managing Global Arrays ..... 16
4.3.1 Iterating Over a Set of Child Nodes ..... 16
4.3.2 Iteration with next() ..... 17
4.3.3 Testing for Child Nodes and Node Values ..... 17

**5 Managing Transactions and Locking with Node.js ..... 21**
5.1 Processing Transactions in Node.js ..... 21
5.2 Concurrency Control with Node.js ..... 22

**6 Native SDK for Node.js Quick Reference ..... 25**
6.1 Module intersystems-iris createConnection() ..... 25
6.1.1 Creating a Connection in Node.js ..... 26
6.2 Class Connection ..... 26
6.2.1 Class Connection Method Details ..... 27
6.3 Class Iris ..... 27
6.3.1 Class Iris Method Details ..... 27
6.4 Class IRISObject ..... 32
6.4.1 IRISObject Constructor ..... 33
6.4.2 IRISObject Method Details ..... 33
6.5 Class Iterator ..... 34

# List of Figures

Figure 3–1: Node.js External Server System ..... 8

# 1

# Introduction to the Native SDK for Node.js

See the Table of Contents for a detailed listing of the subjects covered in this document. See the Node.js Native SDK Quick Reference for a brief description of Native SDK classes and methods.

The InterSystems *Native SDK for Node.js* is a lightweight interface to powerful InterSystems IRIS® resources that were once available only through ObjectScript:

- Call ObjectScript Methods and Functions — call any embedded language classmethod from your Node.js application as easily as you can call native Node.js methods.
- Manipulate Databse Objects — use Node.js proxy objects to control embedded language class instances. Call instance methods and get or set property values as if the instances were native Node.js objects.
- Work with Global Arrays — directly access globals, the tree-based sparse arrays used to implement the InterSystems multidimensional storage model.
- Use InterSystems Transactions and Locking — use Native SDK implementations of embedded language transactions and locking methods to work with InterSystems databases.

**Note:** **Support for Node.js Relational Access**

InterSystems also supports Node.js relational access. The *node-odbc* open source Node.js module is supported on both Windows and UNIX® based systems. See "Support for Node.js Relational Access" in *Using the InterSystems ODBC Driver* for details.

## Native SDKs for other languages

Versions of the Native SDK are also available for Java, .NET, and Python:

- *Using the Native SDK for Java*
- *Using the Native SDK for .NET*
- *Using the Native SDK for Python*

## More information about globals

The following book is highly recommended for developers who want to master the full power of globals:

- *Using Globals* — describes how to use globals in ObjectScript, and provides more information about how multidimensional storage is implemented on the server.

# 2

# Calling Class Methods from Node.js

This section describes methods of class Iris that allow you to call ObjectScript class methods directly from your Node.js application. See the following sections for details and examples:

- Calling Methods from Node.js — demonstrates how to call methods of ObjectScript classes.
- Passing Arguments by Reference — demonstrates how to pass method arguments with the IRISReference class. .

## 2.1 Calling Class Methods from Node.js

The **classMethodValue()** and **classMethodVoid()** methods will work for most purposes, but if a specific return type is needed, the following Iris. typecast methods are also available: **classMethodBoolean()**, **classMethodBytes()**, **classMethodDecimal()**, **classMethodFloat()**, **classMethodIRISList()**, **classMethodInteger()**, **classMethodObject()**, and **classMethodString()**.

These methods all take string arguments for *class_name* and *method_name*, plus `0` or more method arguments.

The code in the following example calls class methods of several datatypes from an ObjectScript test class named User.NativeTest. (see listing "ObjectScript Class User.NativeTest" at the end of this section).

### Node.js calls to ObjectScript class methods

The code in this example calls class methods of each supported datatype from ObjectScript test class User.NativeTest (listed immediately after this example). Assume that variable *irisjs* is a previously defined instance of class Iris and is currently connected to the server (see "Creating a Connection in Node.js").

```
const className = 'User.NativeTest';
let cmValue = "";
let comment = "";

comment = ".cmBoolean() tests whether arguments 2 and 3 are equal: "
cmValue = irisjs.classMethodBoolean(className,'cmBoolean',2,3);
console.log(className + comment + cmValue);

comment = ".cmBytes() returns integer arguments 72,105,33 as a byte array (string value 'Hi!'):
"
cmValue = irisjs.classMethodBytes(className,'cmBytes',72,105,33); //ASCII 'Hi!'
console.log(className + comment + cmValue);

comment = ".cmString() concatenates 'Hello' with argument string 'World': "
cmValue = irisjs.classMethodString(className,'cmString','World');
console.log(className + comment + cmValue);

comment = ".cmLong() returns the sum of arguments 7+8: "
cmValue = irisjs.classMethodInteger(className,'cmLong',7,8);
console.log(className + comment + cmValue);
```

```
  comment = ".cmDouble() multiplies argument 4.5 by 1.5: "
  cmValue = irisjs.classMethodFloat(className,'cmDouble',4.5);
  console.log(className + comment + cmValue);

  comment = ".cmList() returns a $LIST containing arguments 'The answer is ' and 42: "
  cmValue = irisjs.classMethodIRISList(className,"cmList","The answer is ",42);
  console.log(className + comment+cmValue.get(1)+cmValue.get(2) )

  comment = ".cmVoid() assigns argument value 75 to global node ^cmGlobal: "
  try {
    irisjs.kill('cmGlobal') // delete old ^cmGlobal if it exists
    irisjs.classMethodVoid(className,'cmVoid',75);
    cmValue = irisjs.get("cmGlobal");  //get current value of ^cmGlobal
  }
  catch {cmValue = 'method failed.'}
  console.log(className + comment + cmValue);
```

This example demonstrates the classMethodValue() Typecast Methods for each supported type. It omits **classMethodValue()** (which returns an untyped value) and **classMethodObject()** (which is demonstrated in "Controlling Database Objects with Node.js").

### ObjectScript Class User.NativeTest

To run the previous example, this ObjectScript class must be compiled and available on the server:

```
Class User.NativeTest Extends %Persistent
  {

  ClassMethod cmBoolean(cm1 As %Integer, cm2 As %Integer) As %Boolean
  {
     Quit (cm1=cm2)
  }

  ClassMethod cmBytes(cm1 As %Integer, cm2 As %Integer, cm3 As %Integer) As %Binary
  {
     Quit $CHAR(cm1,cm2,cm3)
  }

  ClassMethod cmString(cm1 As %String) As %String
  {
     Quit "Hello "_cm1
  }

  ClassMethod cmLong(cm1 As %Integer, cm2 As %Integer) As %Integer
  {
     Quit cm1+cm2
  }

  ClassMethod cmDouble(cm1 As %Double) As %Double
  {
     Quit cm1 * 1.5
  }

  ClassMethod cmVoid(cm1 As %Integer)
  {
     Set ^cmGlobal=cm1
     Quit
  }

  ClassMethod cmList(cm1 As %String, cm2 As %Integer)
  {
     Set list = $LISTBUILD(cm1,cm2)
     Quit list
  }
}
```

You can test these methods by calling them from the Terminal. For example:

```
USER>write ##class(User.NativeTest).cmString("World")
Hello World
```

**Note:** **Functions and Procedures**

Earlier versions of the Node.js Native SDK provided methods that directly accessed ObjectScript functions and procedures. These methods have been removed due to security concerns. Functions and procedures can still be called indirectly by making them available as methods in an ObjectScript wrapper class.

# 2.2 Passing Arguments by Reference

Most of the classes in the InterSystems Class Library use a calling convention where methods only return a %Status value. The actual results are returned in arguments passed by reference. The Node.js Native SDK supports pass by reference for both methods and functions by assigning the argument value to an instance of class IRISReference and passing that instance as the argument:

```
const IRISNative = require('intersystems-iris')
var ref_object = new IRISNative.IRISReference(null) // set inital value to 0
var status = irisjs.classMethodObject("%SomeClass","SomeMethod", arglist, ref_object);
var returned_value = ref_object.getValue;  // get the method result
```

The following example calls a standard Class Library method:

### Using pass-by-reference arguments

This example calls %SYS.DatabaseQuery.**GetDatabaseFreeSpace()** to get the amount of free space (in MB) available in the iristemp database.

```
  const IRISNative = require('intersystems-iris')
  var freeMB = new IRISNative.IRISReference(null) // set inital value to 0
  const dir = "C:/InterSystems/IRIS/mgr/iristemp" // directory to be tested
  var value = "error"
  var status = 0

  console.log("Type of freeMB = "+ (typeof freeMB) + ", value = " + freeMB.getValue())
  try{
    console.log("Calling %SYS.DatabaseQuery.GetDatabaseFreeSpace()... ")
    status = irisjs.classMethodObject("%SYS.DatabaseQuery","GetDatabaseFreeSpace",dir,freeMB)
    value = freeMB.getValue()
  } catch {
    console.log("Call to class method GetDatabaseFreeSpace() returned error:")
  }
  console.log("(status=" + status + ") Free space in " + dir + " = " + value + "MB\n")
}
```

prints:

```
Type of freeMB=str(type(freeMB)), value=null
Calling %SYS.DatabaseQuery.GetDatabaseFreeSpace()...
(status=1) Free space in C:/InterSystems/IRIS/mgr/iristemp = 10MB
```

# 3

# Controlling Database Objects from Node.js

The Native SDK works together with InterSystems External Servers, allowing your external Node.js application to control instances of database classes written in either ObjectScript or Embedded Python. Native SDK *inverse proxy objects* can use External Server connections to create a target database object, call the target's instance methods, and get or set property values as easily as if the target were a native Node.js object.

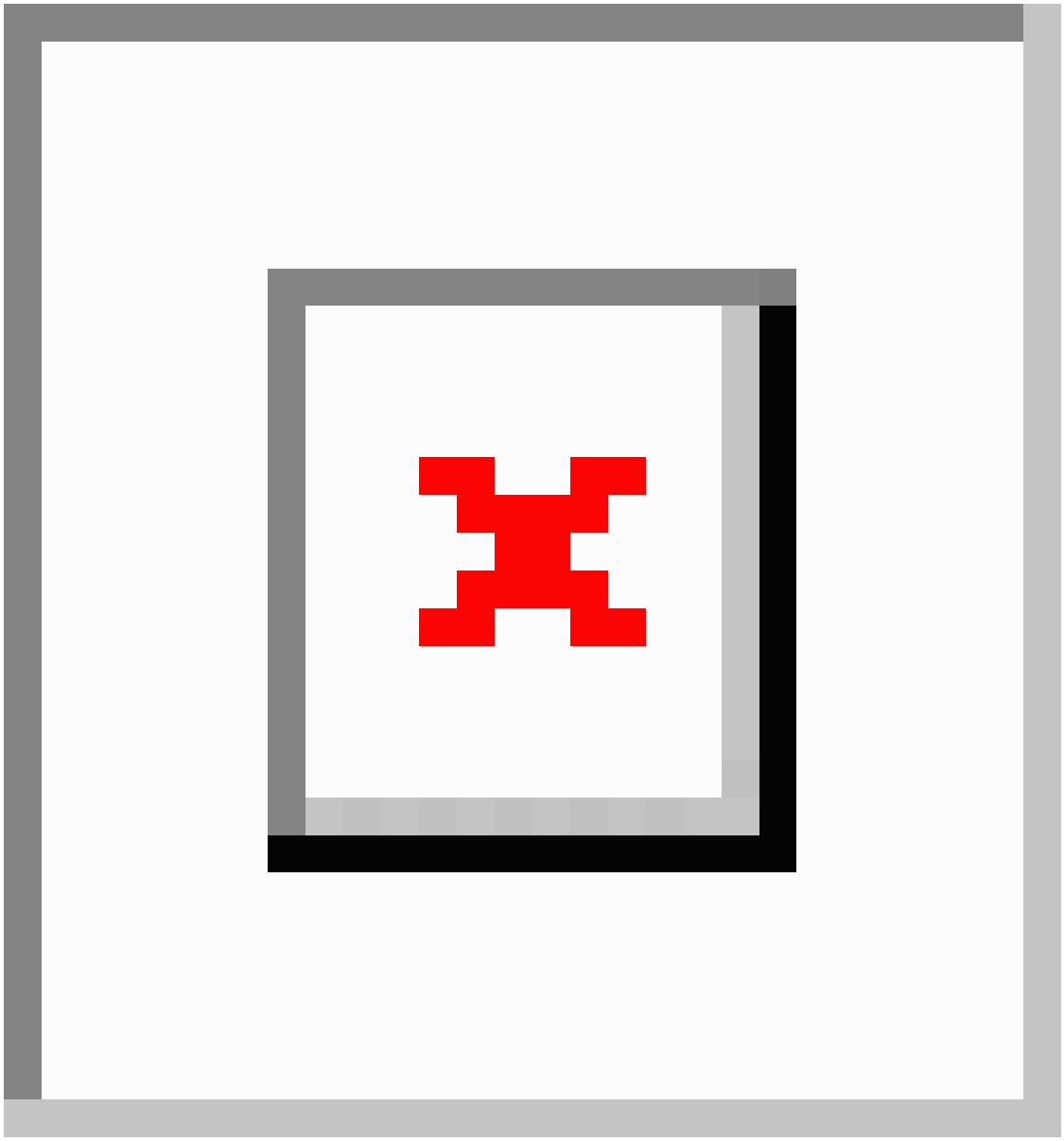
This section covers the following topics:

- Introducing the Node.js External Server — provides a brief overview of External Servers.
- Creating Node.js Inverse Proxy Objects — describes methods used to create inverse proxy objects.
- Controlling Database Objects with IRISObject — demonstrates how inverse proxy objects are used.

## 3.1 Introducing the Node.js External Server

The Node.js External Server allows InterSystems IRIS embedded language objects and Node.js Native SDK objects to interact freely, using the same connection and working together in the same context (database, session, and transaction). External Server architecture is described in detail in *Using InterSystems External Servers*, but for the purposes of this discussion you can think of it as a simple black box connecting proxy objects on one side to target objects on the other:

*Figure 3–1: Node.js External Server System*



As the diagram shows, a *forward proxy* is a database object that controls an external Node.js target object. The corresponding Native SDK object is an *inverse proxy* that controls a database target object from an external Node.js application, inverting the normal flow of control.

# 3.2 Creating Node.js Inverse Proxy Objects

You can create an inverse proxy object by obtaining the OREF of a database class instance (for example, by calling the **%New()** method of an ObjectScript class). The Iris.**classMethodObject()** method will return an IRISObject instance if the call obtains a valid OREF. The following example uses Iris.**classMethodObject()** to create an inverse proxy object:

### Creating an instance of IRISObject

```
var proxy = irisjs.classMethodObject("User.TestInverse","%New");
```

- **classMethodObject()** calls the **%New()** method of an ObjectScript class named User.TestInverse
- The call to **%New()** creates a database instance of User.TestInverse.
- **classMethodObject()** returns inverse proxy object *proxy*, which is an instance of IRISObject mapped to the database instance.

This example assumes that *irisjs* is a connected instance of Iris (see "Creating a Connection in Node.js"). See "Calling Class Methods from Node.js" for more information on how to call class methods.

The following section demonstrates how *proxy* can be used to access methods and properties of the ObjectScript User.TestInverse instance.

# 3.3 Controlling Database Objects with IRISObject

The iris.IRISObject class provides several methods to control the database target object: **invoke()** and **invokeVoid()** call an instance method with or without a return value, and accessors **get()** and **set()** get and set a property value. .

This example uses an ObjectScript class named User.TestInverse, which includes declarations for methods **initialize()** and **add()**, and property *name*:

### ObjectScript sample class TestInverse

```
Class User.TestInverse Extends %Persistent {
  Method initialize(initialVal As %String = "no name") {
    set ..name = initialVal
    return 0
  }
  Method add(val1 As %Integer, val2 As %Integer) As %Integer {
   return val1 + val2
  }
  Property name As %String;
}
```

The first line of the following example creates a new instance of User.TestInverse and returns inverse proxy object *proxy*, which is mapped to the database instance. The rest of the code uses *proxy* to access the target instance. Assume that a connected instance of IRIS named *irisjs* already exists (see "Creating a Connection in Node.js").

### Using inverse proxy object methods in Node.js

```
  // Create an instance of User.TestInverse and return an inverse proxy object for it
  var proxy = irisjs.classMethodObject("User.TestInverse","%New");

  // instance method proxy.initialize() is called with one argument, returning nothing.
  proxy.invokeVoid('initialize', 'George');
  console.log("Current name is "+ proxy.get("name"));   // display the initialized property
value

  // instance method proxy.add() is called with two arguments, returning an int value.
  console.log("Sum of 2 plus 3 is " + proxy.invoke("add",2,3));

  // The value of property proxy.name is displayed, changed, and displayed again.
  proxy.set("name", "Einstein, Albert");  // sets the property to "Einstein, Albert"
  console.log("New name is "+ proxy.get("name"));   // display the new property value
```

This example uses the following methods to access methods and properties of the User.TestInverse instance:

- IRISObject.**invokeVoid()** invokes the **initialize()** instance method, which initializes a property but does not return a value.
- IRISObject.**invoke()** invokes instance method **add()**, which accepts two integer arguments and returns the sum as an integer.

- IRISObject.**set()** sets the *name* property to a new value.
- IRISObject.**get()** returns the value of property *name*.

This example used the variant **get()**, and **invoke()** methods, but the IRISObject class also provides datatype-specific Typecast Methods for supported datatypes:

### IRISObject get() typecast methods

In addition to the variant **get()** method, IRISObject provides the following IRISObject. typecast methods: **getBytes()**, **getDecimal() getFloat() getInteger()**, **getString()**, **getIRISList()**, and **getObject()**.

### IRISObject invoke() typecast methods

In addition to **invoke()** and **invokeVoid()**, IRISObject provides the following IRISObject. typecast methods: **invokeBytes()**, **invokeDecimal()**, **invokeInteger()**, **invokeString()**, **invokeIRISList()**, and **invokeObject()**.

All of the **invoke()** methods take a string argument for *methodName* plus `0` or more method arguments. The arguments may be either Node.js objects, or values of supported datatypes. Database proxies will be generated for arguments that are not supported types.

**invoke()** can only call instance methods of an IRISObject instance. See "Calling Database Methods and Functions from Node.js" for information on how to call class methods.

# 4

# Accessing Global Arrays with Node.js

The *Native SDK for Node.js* provides mechanisms for working with global arrays. The following topics are covered here:

- Introduction to Global Arrays — introduces global array concepts and provides a simple demonstration of how the Native SDK is used.
- Fundamental Node Operations — demonstrates how use **set()**, **get()**, and **kill()** to create, access, and delete nodes in a global array.
- Managing Global Arrays — describes how to create and iterate over complex global arrays.

**Note:** **Creating a Database Connection in Node.js**

The examples in this chapter assume that an Iris object named *irisjs* already exists and is connected to the server. The following code was used to create and connect it:

```
const IRISNative = require('intersystems-iris')

// Open a connection to the server and create an Iris object
  let connectionInfo = {
    host: '127.0.0.1',
    port: 51773,
    ns: 'USER',
    user: '_SYSTEM',
    pwd: 'SYS'
  };
const conn = IRISNative.createConnection(connectionInfo);
const irisjs = conn.createIris();
```

For more information, see the Quick Reference entries for **createConnection()** and **createIris()**.

## 4.1 Introduction to Global Arrays

A global array, like all sparse arrays, is a tree structure rather than a sequential list. The basic concept behind global arrays can be illustrated by analogy to a file structure. Each *directory* in the tree is uniquely identified by a *path* composed of a *root directory* identifier followed by a series of *subdirectory* identifiers, and any directory may or may not contain *data*.

Global arrays work the same way: each *node* in the tree is uniquely identified by a *node address* composed of a *global name* identifier and a series of *subscript* identifiers, and a node may or may not contain a *value*. For example, here is a global array consisting of six nodes, two of which contain values:

```
root -->|--> foo --> SubFoo='A'
        |--> bar --> lowbar --> UnderBar=123
```

Values could be stored in the other possible node addresses (for example, root or root->bar), but no resources are wasted if those node addresses are *valueless*. Unlike a directory structure, all nodes in a global array *must* have either a value or a subnode with a value. In InterSystems ObjectScript globals notation, the two nodes with values would be:

```
root('foo','SubFoo')
root('bar','lowbar','UnderBar')
```

In this notation, the global name (`root`) is followed by a comma-delimited *subscript list* in parentheses. Together, they specify the entire *node address* of the node.

This global array could be created by two calls to the Native SDK **set()** method. The first argument is the value to be assigned, and the rest of the arguments specify the node address:

```
irisjs.set('A', 'root', 'foo', 'SubFoo');
irisjs.set(123, 'root', 'bar', 'lowbar', 'UnderBar');
```

Global array `root` is does not exist until the first call assigns value `'A'` to node *root('foo','SubFoo')*. Nodes can be created in any order, and with any set of subscripts. The same global array would be created if we reversed the order of these two calls. The valueless nodes are created automatically, and will be deleted automatically when no longer needed.

The Native SDK code to create this array is demonstrated in the following example. An IRISConnection object establishes a connection to the server. The connection will be used by an instance of class Iris named *irisjs*. Native SDK methods are then used to create a global array, read the resulting persistent values from the database, and delete the global array.

### The NativeDemo Program

```
// Import the Native SDK module
const IRISNative = require('intersystems-iris')

// Open a connection to the server and create an Iris object
  let connectionInfo = {
    host: '127.0.0.1',
    port: 51773,
    ns: 'USER',
    user: '_SYSTEM',
    pwd: 'SYS'
  };
const conn = IRISNative.createConnection(connectionInfo);
const irisjs = conn.createIris();

// Create a global array in the USER namespace on the server
  irisjs.set('A', 'root', 'foo', 'SubFoo');
  irisjs.set(123, 'root', 'bar', 'lowbar', 'UnderBar');

// Read the values from the database and print them
  let subfoo = irisjs.get('root', 'foo', 'SubFoo')
  let underbar = irisjs.get('root', 'bar', 'lowbar', 'UnderBar')
  console.log('Created two values: ');
  console.log('   root("foo","SubFoo")=' + subfoo);
  console.log('   root("bar","lowbar","UnderBar")=' + underbar);

// Delete the global array and terminate
  irisjs.kill('root'); // delete entire global array
  conn.close();
```

NativeDemo prints the following lines:

```
Created two values:
   root("foo","SubFoo")="A"
   root("bar","lowbar","UnderBar")=123
```

In this example, method **createConnection()** defines a Connection object named *conn* that provides a connection to the database associated with the USER namespace. Native SDK methods perform the following actions:

- Connection.**createIRIS()** creates a new instance of Iris named *irisjs*, which will access the database through server connection *conn*.
- Iris.**set()** creates new persistent nodes in database namespace USER.

- Iris.**get()** queries the database and returns the values of the specified nodes.
- Iris.**kill()** deletes the specified root node and all of its subnodes from the database.

The next chapter provides detailed explanations and examples for all of these methods.

## 4.1.1 Glossary of Global Array Terms

See the previous section for an overview of the concepts listed here. Examples in this glossary will refer to the global array structure listed below. The *Legs* global array has ten nodes and three node levels. Seven of the ten nodes contain values:

```
Legs                        // root node, valueless, 3 child nodes
  fish = 0                  // level 1 node, value=0
  mammal                    // level 1 node, valueless
    human = 2               // level 2 node, value=2
    dog = 4                 // level 2 node, value=4
  bug                       // level 1 node, valueless, 3 child nodes
    insect = 6              // level 2 node, value=6
    spider = 8              // level 2 node, value=8
    millipede = Diplopoda   // level 2 node, value='Diplopoda', 1 child node
      centipede = 100       // level 3 node, value=100
```

### Child node

The nodes immediately under a given parent node. The address of a child node is specified by adding exactly one subscript to the end of the parent subscript list. For example, parent node *Legs('mammal')* has child nodes *Legs('mammal','human')* and *Legs('mammal','dog')*.

### Global name

The identifier for the root node is also the name of the entire global array. For example, root node identifier `Legs` is the global name of global array *Legs*. Unlike subscripts, global names can only consist of letters, numbers, and periods (see Global Naming Rules).

### Node

An element of a global array, uniquely identified by a namespace consisting of a global name and an arbitrary number of subscript identifiers. A node must either contain a value, have child nodes, or both.

### Node level

The number of subscripts in the node address. A ‘level 2 node’ is just another way of saying ‘a node with two subscripts’. For example, *Legs('mammal','dog')* is a level 2 node. It is two levels under root node *Legs* and one level under *Legs('mammal')*.

### Node address

The complete namespace of a node, consisting of the global name and all subscripts. For example, node address *Legs('fish')* consists of root node identifier `Legs` plus a list containing one subscript, `'fish'`. Depending on context, *Legs* (with no subscript list) can refer to either the root node address or the entire global array.

### Root node

The unsubscripted node at the base of the global array tree. The identifier for a root node is its global name with no subscripts.

### Subnode

All descendants of a given node are referred to as *subnodes* of that node. For example, node *Legs('bug')* has four different subnodes on two levels. All nine subscripted nodes are subnodes of root node *Legs*.

#### Subscript / Subscript list

All nodes under the root node are addressed by specifying the global name and a list of one or more subscript identifiers. Subscripts can be strings or numbers. The global name plus the subscript list is the node address.

#### Target address

Many Native SDK methods require you to specify a valid node address that does not necessarily point to an existing node. For example, the **set()** method takes a *value* argument and a target address, and stores the value at that address. If no node exists at the target address, a new node is created at that address.

#### Value

A node can contain a value of any supported type. A node with no child nodes must contain a value; a node that has child nodes can be valueless.

#### Valueless node

A node must either contain data, have child nodes, or both. A node that has child nodes but does not contain data is called a valueless node. Valueless nodes only exist as pointers to lower level nodes.

### 4.1.2 Global Naming Rules

Global names and subscripts obey the following rules:

- The length of a node address (totaling the length of the global name and all subscripts) can be up to 511 characters. (Some typed characters may count as more than one encoded character for this limit. For more information, see "Maximum Length of a Global Reference" in *Using Globals*).
- A global name can include letters, numbers, and periods (`'.'`), and can have a length of up to 31 significant characters. It must begin with a letter, and must not end with a period.
- A subscript can be a string, an integer, or a number. String subscripts are case-sensitive, and can contain any character (including non-printing characters). Subscript length is restricted only by the maximum length of a node address.

## 4.2 Fundamental Node Operations

This section demonstrates how to use the **set()**, **get()**, and **kill()** methods to create, access, and delete nodes. These methods have the following signatures:

```
set (value, globalName, subscripts)
get (globalName, subscripts)
kill (globalName, subscripts)
```

- *value* is the item to be stored.
- *globalName* can only include letters, numbers, and periods (`'.'`), must begin with a letter, and cannot end with a period.
- *subscripts* can be strings, integers, or floating point numbers. A string subscript is case-sensitive and can include non-printing characters.

All of the examples in this section assume that a connected instance of class Iris named *irisjs* already exists (see "Creating a Connection in Node.js").

### Setting and changing node values

Iris.**set()** takes a *value* argument and stores the value at the specified address.

If no node exists at that address, a new one is created.

The **set()** method can assign values of any supported datatype. In the following example, the first call to **set()** creates a new node at subnode address *myGlobal('A')* and sets the value of the node to string `'first'`. The second call changes the value of the subnode, replacing it with integer `1`.

```
irisjs.set('first', 'myGlobal', 'A');     // create node myGlobal('A') = 'first'
irisjs.set(1, 'myGlobal', 'A');   // change value of myGlobal('A') to 1.
```

**set()** is a polymorphic accessor that can create and change values of any supported datatype, as demonstrated in this example.

### Retrieving node values with get()

Iris.**get()** takes *globalname* and **subscripts* arguments and returns the value stored at the specified node address, or None if there is no value at that address.

```
irisjs.set(23,'myGlobal','A');
value_of_A = irisjs.get('myGlobal','A')
```

The **get()** method returns an untyped value. To return a specific datatype, use one of the IRIS.get() typecast methods. The following methods are available: **getBoolean()**, **getBytes()**, **getDecimal() getFloat() getInteger()**, **getString()**, **getIRISList()**, and **getObject()**.

### Deleting a node or group of nodes

Iris.**kill()** — deletes the specified node and all of its subnodes. The entire global array will be deleted if the root node is deleted or if all nodes with values are deleted.

Global array *myGlobal* initially contains the following nodes:

```
myGlobal = <valueless node>
  myGlobal('A') = 0
    myGlobal('A',1) = 0
    myGlobal('A',2) = 0
  myGlobal('B') = <valueless node>
    myGlobal('B',1) = 0
```

This example will delete the global array by calling **kill()** on two of its subnodes. The first call will delete node *myGlobal('A')* and both of its subnodes:

```
irisjs.kill('myGlobal','A')    // also kills myGlobal('A',1) and myGlobal('A',2)
```

The second call deletes the last remaining subnode with a value, killing the entire global array:

```
irisjs.kill('myGlobal','B',1)  // deletes last value in global array myGlobal
```

- The parent node, *myGlobal('B')*, is deleted because it is valueless and now has no subnodes.
- Root node *myGlobal* is valueless and now has no subnodes, so the entire global array is deleted from the database.

# 4.3 Managing Global Arrays

The Native SDK provides ways to iterate over part or all of a global array. The following topics describe the various iteration methods:

- Iterating Over a Set of Child Nodes — describes how to iterate over all child nodes under a given parent node.
- Iteration with next() — describes methods that provide more control over iteration.
- Testing for Child Nodes and Node Values — describes how to find all subnodes regardless of node level, and identify which nodes have values.

## 4.3.1 Iterating Over a Set of Child Nodes

Child nodes are sets of nodes immediately under the same parent node. Any child node address can be defined by appending one subscript to the subscript list of the parent. For example, the following global array has four child nodes under parent node *heroes('dogs')*:

### The heroes global array

This global array uses the names of several heroic dogs (plus a reckless boy and a pioneering sheep) as subscripts. The values are birth years.

```
heroes                                // root node, no value, 2 child nodes
  heroes('dogs')                             // level 1, no value, 4 child nodes
    heroes('dogs','Balto') = 1919               // level 2, value=1919
    heroes('dogs','Hachiko') = 1923             // level 2, value=1923
    heroes('dogs','Lassie') = 1940              // level 2, value=1940, 1 child
      heroes('dogs','Lassie','Timmy') = 1954       // level 3, value=1954
    heroes('dogs','Whitefang') = 1906           // level 2, value=1906
  heroes('sheep')                            // level 1, no value, 1 child
    heroes('sheep','Dolly') = 1996              // level 2, value=1996
```

The following methods are used to create an iterator, define the direction of iteration, and set the starting point of the search:

- Iris.**iterator()** returns an instance of Iterator for the child nodes of the specified target node.
- Iterator.**reversed()** — toggles direction of iteration between forward and reverse collation order.
- Iterator.**startFrom()** sets the iterator's starting position to the specified subscript. The subscript is an arbitrary starting point, and does not have to address an existing node.

### Read child node values in reverse order

The following code iterates over child nodes of *heroes('dogs')* in reverse collation order, starting with subscript V:

```
// Iterate in reverse, seeking nodes lower than heroes('dogs','V') in collation order
  let iterDogs = irisjs.iterator('heroes','dogs').reversed().startFrom('V');

  let output = '\nDog birth years: ';
  for ([key,value] of iterDogs) {
    output += key + ':' + value + '  ';
  };
  console.log(output);
```

This code prints the following output:

```
  Dog birth years: Lassie:1940  Hachiko:1923  Balto:1919
```

In this example, two subnodes of *heroes('dogs')* are ignored:

- Child node *heroes('dogs','Whitefang')* will not be found because it is outside of the search range (`Whitefang` is higher than `V` in collation order).
- Level 3 node *heroes('dogs','Lassie','Timmy')* will not be found because it is a child of `Lassie`, not `dogs`.

See the last section in this chapter (“Testing for Child Nodes and Node Values”) for a discussion of how to iterate over multiple node levels.

**Note:** **Collation Order**

The order in which nodes are retrieved depends on the *collation order* of the subscripts. When a node is created, it is automatically stored it in the collation order specified by the storage definition. In this example, the child nodes of *heroes('dogs')* would be stored in the order shown (`Balto`, `Hachiko`, `Lassie`, `Whitefang`) regardless of the order in which they were created. For more information, see “Collation of Global Nodes” in *Using Globals*.

## 4.3.2 Iteration with next()

The Native SDK also supports the standard **next()** and return type iterator methods:

- Iterator.**next()** — positions the iterator at the next child node (if one exists) and returns an array containing properties `done` and `value`. The `done` property will be `false` if there are no more nodes. When an iterator is created, it defaults to the **entries()** return type.
- Iterator.**entries()** — sets return type to an array containing both the key (the top level subscript of the child node) and the node value. For example, the returned value for node *heroes(,'dogs','Balto')* would be `['Balto',1919]`.
- Iterator.**keys()** — sets return type to return only the key (the top level subscript).
- Iterator.**values()** — sets return type to return only the node value.

In the following example, each call to the **next()** method sets variable *iter* to an array containing the current values for the iterator `done` and `value` properties. Since the **keys()** method was called when the iterator was created, the `value` property will contain only the key (top level subscript) for the current child node of *heroes('dogs')*.

### Use next() to list the subscripts under node heroes('dogs')

```
// Get a list of child subscripts under node heroes('dogs')
  let iterDogs = irisjs.iterator('heroes','dogs').keys();
  let output = "\nSubscripts under node heroes('dogs'): ";

  let iter = iterDogs.next();
  while (!iter.done) {
    output += iter.value + ' ';
    iter = iterDogs.next();
  }
  console.log(output);
```

This code prints the following output:

```
Subscripts under node heroes('dogs'): Balto Hachiko Lassie Whitefang
```

## 4.3.3 Testing for Child Nodes and Node Values

In the previous examples, the scope of the search is restricted to child nodes of *heroes('dogs')*. The iterator fails to find two values in global array heroes because they are under different parents:

- Level 3 node *heroes('dogs','Lassie','Timmy')* will not be found because it is a child of `Lassie`, not `dogs`.
- Level 2 node *heroes('sheep','Dolly')* is not found because it is a child of `sheep`, not `dogs`.

To search the entire global array, we need to find all of the nodes that have child nodes, and create an iterator for each set of child nodes. The **isDefined()** method provides the necessary information:

- Iris.**isDefined()** — can be used to determine if a node has a value, a subnode, or both. It returns one of the following values:
  - `0` — the specified node does not exist
  - `1` — the node exists and has a value
  - `10` — the node is valueless but has a child node
  - `11` — the node has both a value and a child node

  The returned value can be used to determine several useful boolean values:

```
let exists = (irisjs.isDefined(root,subscripts) > 0);      // returns 1, 10, or 11
let hasValue = (irisjs.isDefined(root,subscripts)%10 > 0); // returns 1 or 11
let hasChild = (irisjs.isDefined(root,subscripts) > 9);    // returns 10 or 11
```

The following example consists of two methods:

- **findAllHeroes()** iterates over child nodes of the current node, and calls **testNode()** for each node. Whenever **testNode()** indicates that the current node has child nodes, **findAllHeroes()** creates a new iterator for the next level of child nodes.
- **testNode()** will be called for each node in the *heroes* global array. It calls **isDefined()** on the current node, and returns a boolean value indicating whether the node has child nodes. It also prints node information for each node.

### Method findAllHeroes()

This example processes a known structure, and traverses the various levels with simple nested calls. In the less common case where a structure has an arbitrary number of levels, a recursive algorithm could be used.

```
function findAllHeroes() {
  const root = 'heroes';
  console.log('List all subnodes of root node '+root+':\n'+root)
  let iterRoot = irisjs.iterator(root);
  let hasChild = false;

  // Iterate over children of root node heroes
  for ([sub1,value] of iterRoot) {
    hasChild = testNode(value,root,sub1);

    // Iterate over children of heroes(sub1)
    if (hasChild) {
      let iterOne = irisjs.iterator(root,sub1);
      for ([sub2,value] of iterOne) {
        hasChild = testNode(value,root,sub1,sub2);

        // Iterate over children of heroes(sub1,sub2)
        if (hasChild) {
          let iterTwo = irisjs.iterator(root,sub1,sub2);
          for ([sub3,value] of iterTwo) {
            testNode(value,root,sub1,sub2,sub3); //no child nodes below level 3
          }
        } //end level 2
      }
    } //end level 1
  } // end main loop
} // end findAllHeroes()
```

### Method testNode()

```
function testNode(value, root, ...subs) {

// Test for values and child nodes
  let state = irisjs.isDefined(root,...subs);
  let hasValue = (state%10 > 0); // has value if state is 1 or 11
  let hasChild = (state > 9);    // has child if state is 10 or 11

// format the node address output string
```

```
    let subList = Array.from(subs);
    let level = subList.length-1;
    let indent = '  ' + String('      ').slice(0,(level*2));
    let address = indent + root+'(' + subList.join() + ')';

  // Add node value to string and note special cases
    if (hasValue) { // ignore valueless nodes
      address += ' = ' + value;
      for (name of ['Timmy','Dolly']) {
        if (name == subList[level]) {
          address += ' (not a dog!)'
        }
      }
    }
    console.log(address);
    return hasChild;
  }
}
```

This method will write the following lines:

```
List all subnodes of root node heroes:
heroes
  heroes(dogs)
    heroes(dogs,Balto) = 1919
    heroes(dogs,Hachiko) = 1923
    heroes(dogs,Lassie) = 1940
      heroes(dogs,Lassie,Timmy) = 1954  (not a dog!)
    heroes(dogs,Whitefang) = 1906
  heroes(sheep)
    heroes(sheep,Dolly) = 1996  (not a dog!)
```

The output of **testNodes()** includes some nodes that were not found in previous examples because they are not child nodes of *heroes('dogs')*:

- *heroes('dogs','Lassie','Timmy')* is a child of `Lassie`, not `dogs`.
- *heroes('sheep','Dolly')* is a child of `sheep`, not `dogs`.

# 5

# Managing Transactions and Locking with Node.js

The *Native SDK for Node.js* provides transaction and locking methods that use the InterSystems transaction model, as described in the following sections:

- Processing Transactions — describes how transactions are started, nested, rolled back, and committed.
- Concurrency Control — describes how to use the various lock methods.

For information on the InterSystems transaction model, see "Transaction Processing" in *Using ObjectScript*.

## 5.1 Processing Transactions in Node.js

The Iris class provides the following methods for transaction processing:

- Iris.**tCommit()** — commits one level of transaction.
- Iris.**tStart()** — starts a transaction (which may be a nested transaction).
- Iris.**getTLevel()** — returns an int value indicating the current transaction level (0 if not in a transaction).
- Iris.**increment()** — increments or decrements a node value without locking the node.
- Iris.**tRollback()** — rolls back all open transactions in the session.
- Iris.**tRollbackOne()** — rolls back the current level transaction only. If this is a nested transaction, any higher-level transactions will not be rolled back.

The following example starts three levels of nested transaction, storing a different global node value at each transaction level. All three nodes are printed to prove that they have values. The example then rolls back the second and third levels and commits the first level. All three nodes are printed again to prove that only the first node still has a value.

#### *Controlling Transactions*: Creating and rolling back three levels of nested transaction

Assume that *irisjs* is a connected instance of class Iris (see "Creating a Connection in Node.js").

```
const node = 'myGlobal';
console.log('Set three values in three different transaction levels:');
for (let i=1; i<4; i++) {
  irisjs.tStart();
  let lvl = irisjs.getTLevel()
  irisjs.set(('Value'+lvl), node, lvl);
```

```
      let val = '<valueless>'
      if (irisjs.isDefined(node,lvl)%10 > 0) val = irisjs.get(node,lvl);
      console.log('  ' + node + '(' + i + ') = ' + val + ' (tLevel is ' + lvl + ')');
    }
// Prints: Set three values in three different transaction levels:
//         myGlobal(1) = Value1 (tLevel is 1)
//         myGlobal(2) = Value2 (tLevel is 2)
//         myGlobal(3) = Value3 (tLevel is 3)

  console.log('Roll back two levels and commit the level 1 transaction:');
  let act = ['  tRollbackOne','  tRollbackOne','       tCommit'];
  for (let i=3; i>0; i--) {
    if (i>1) {irisjs.tRollbackOne();} else {irisjs.tCommit();}
    let val = '<valueless>'
    if (irisjs.isDefined(node,i)%10 > 0) val = irisjs.getString(node,i);
    console.log(act[3-i]+' (tLevel='+irisjs.getTLevel()+'): '+node+'('+i+') = '+val);
  }

// Prints: Roll back two levels and commit the level 1 transaction:
//           tRollbackOne (tLevel=2): myGlobal(3) = <valueless>
//           tRollbackOne (tLevel=1): myGlobal(2) = <valueless>
//                tCommit (tLevel=0): myGlobal(1) = Value1
```

**CAUTION:** When a transaction that uses **increment()** is rolled back (with either **tRollback()** or **tRollbackOne()**), counter values are ignored. The counter variables are not decremented because the resulting counter value may not be valid. Such a rollback could be disastrous for other transactions that use the same counter.

# 5.2 Concurrency Control with Node.js

Concurrency control is a vital feature of multi-process systems such as InterSystems IRIS. It provides the ability to lock specific elements of data, preventing the corruption that would result from different processes changing the same element at the same time. The Native SDK transaction model provides a set of locking methods that correspond to ObjectScript commands (see "LOCK" in the *ObjectScript Reference*).

The following methods of class Iris are used to acquire and release locks:

- Iris.**lock()** — locks the node specified by the *lockReference* and **subscripts* arguments. This method will time out after a predefined interval if the lock cannot be acquired.
- Iris.**unlock()** — releases the lock on the node specified by the *lockReference* and **subscripts* arguments.
- Iris.**releaseAllLocks()** — releases all locks currently held by this connection.

*parameters:*

```
lock(lockMode, timeout, lockReference, ...subscript)
unlock(lockMode, lockReference, ...subscript)
releaseAllLocks()
```

- *lockMode* — string specifying how to handle any previously held locks. Valid arguments are, `S` for shared lock, `E` for escalating lock, or `SE` for shared and escalating. Default is empty string (exclusive and non-escalating).
- *timeout* — number of seconds to wait before timing out when attempting to acquire a lock.
- *lockReference* — string starting with a circumflex (`^`) followed by the global name (for example, `^myGlobal`, *not* just `myGlobal`).

  *Important:* the `lockReference` parameter *must* be prefixed by a circumflex, unlike the `globalName` parameter used by most methods. Only **lock()** and **unlock()** use `lockReference` instead of `globalName`.
- *subscripts* — zero or more subscripts specifying the node to be locked or unlocked.

In addition to these methods, the Connection.**close()** method will release all locks and other connection resources.

**Tip:** You can use the Management Portal to examine locks. Go to System Operation > View Locks to see a list of the locked items on your system.

**Note:** A detailed discussion of concurrency control is beyond the scope of this document. See the following articles for more information on this subject:

- "Transaction Processing" and "Lock Management" in *Using ObjectScript*
- "Locking and Concurrency Control" in the *Orientation Guide for Server-Side Programming*
- "LOCK" in the *ObjectScript Reference*

# 6

# Native SDK for Node.js Quick Reference

This is a quick reference for the InterSystems IRIS Native SDK for Node.js, providing information on the following classes:

- Module intersystems-iris
  - intersystems-iris createConnection() — creates an instance of Connection.
  - Class Connection connects an Iris instance to the database.
  - Class Iris is the main entry point for the Native SDK
  - Class IRISObject provides methods for External Server inverse proxy objects.
  - Class Iterator provides methods to navigate a global array.

### Typecast Methods and Supported Datatypes

Many Native SDK methods have built-in support for typecasting. Each of these classes has a generic method that returns a default value, plus a set of typecast methods that work exactly like the generic method but also cast the return value to a supported datatype: boolean, Bytes (an ArrayBuffer), number (plus casts to Integer, Float, and Decimal), string, Object, and IRISList. For example, Iris.**get()** has corresponding typecast methods **getBoolean()**, **getBytes()**, and so forth.

To avoid lengthening this quick reference with dozens of nearly identical listings, all typecast methods are listed collectively after the generic version (for example, Iris.**get()** is followed by a section listing "Iris.get() Typecast Methods").

Sets of typecast methods are available for Iris.classMethodValue(), Iris.get(), IRISObject.get(), and IRISObject.invoke().

## 6.1 Module intersystems-iris createConnection()

### createConnection()

intersystems-iris **createConnection()** returns a new Connection object and attempts to create a new connection to the database. The object will be open if the connection was successful. Throws an exception if a connection cannot be established.

```
(static) createConnection(connectionInfo)   {"intersystems-iris".Connection}
```

The *host*, *port*, *ns*, *timeout*, and *logfile* from the last successful connection attempt are saved as properties of the connection object.

*parameters:*

- `connectionInfo` — Object containing connection properties. Valid properties are:

- `host` — (required) string containing the address of the host machine
- `port` — (required) integer specifying the port number
- `ns` — (required) string specifying the database namespace
- `user` — string specifying the user name
- `pwd` — string specifying the password
- `sharedmemory` — boolean indicating whether to use shared memory if available (default is true). Specify `False` to force a connection over TCP/IP.
- `timeout` — integer specifying the number of milliseconds to wait before the connection attempt times out (default is 10000).
- `logfile` — string specifying the full path and name of a log file for this connection. If not specified, no log file will be written.
- `sslcontext` — path to the file that contains the sslcontext configuration required for a secure connection.

The following property can be used in place of the `host`, `port`, and `ns` properties:

- `connectionstr` — string of the form *`host:port/ns`*

### 6.1.1 Creating a Connection in Node.js

The following code fragment defines all properties of *connectionInfo* and creates an instance of Connection named *conn*. The *host*, *port*, *ns*, *timeout*, and *logfile* from the last successful connection attempt are saved as properties of the connection object.

```
const IRISNative = require('intersystems-iris')

let connectionInfo = {
  host: '127.0.0.1',
  port: 51773,
  ns: 'USER',
  user: '_SYSTEM',
  pwd: 'SYS',
  sharedmemory: true
  timeout: 5,
  logfile: 'C:\temp\mylogfile.log'
};

const conn = IRISNative.createConnection(connectionInfo);

irisjs = conn.createIRIS()
```

The *conn* object is used to create an instance of class Iris named *irisjs*. Most examples in this document will assume that a connected instance of *irisjs* already exists.

## 6.2 Class Connection

The Connection class encapsulates a connection to the server. Instances of Connection are created and connected to the server by intersystems-iris method **createConnection()**. The *hostname*, *port*, *namespace*, *timeout*, and *logfile* from the last successful connection attempt are saved as properties of the connection object. See the previous section (“Module intersystems-iris createConnection()”) for details and examples.

## 6.2.1 Class Connection Method Details

### close()

Connection.**close()** closes the connection if it is open. Does nothing if the connection is already closed.

```
close()
```

### createIRIS()

Connection.**createIris()** returns a new instance of Iris that uses this Connection. Throws an exception if the connection is closed.

```
createIris()   {"intersystems-iris".Iris}
```

### isClosed()

Connection.**isClosed()** returns `True` if the connection was successful, or `False` otherwise.

```
isClosed()   {boolean}
```

### isUsingSharedMemory()

Connection.**isUsingSharedMemory()** returns `True` if the connection is open and using shared memory.

```
isUsingSharedMemory()   {boolean}
```

# 6.3 Class Iris

To use the Native SDK, your application must create an instance of Iris with a connection to the database. Instances of Iris are created by calling Connection.**createIris()**.

## 6.3.1 Class Iris Method Details

### classMethodValue()

Iris.**classMethodValue()** calls a class method, passing zero or more arguments and returns the value as a type corresponding to the datatype of the ObjectScript return value. See "Calling Class Methods from Node.js" for details and examples.

This method has a corresponding set of typecast methods (listed below).

```
classMethodValue(className, methodName, ...args)   {any}
```

*parameters:*

- `className` — fully qualified name of the class to which the called method belongs.
- `methodName` — name of the class method.
- `...arguments` — zero or more method arguments of supported types. Number, string, and IRISList return values are projected as literals. All other values are projected as proxy objects.

This method has a corresponding set of typecast methods (listed below). Also see **classMethodVoid()**, which is used to call methods that do not return a value.

### Iris.classMethodValue() Typecast Methods

All of the Iris.**classMethodValue()** typecast methods listed below work exactly like Iris.**classMethodValue()**, but also cast the return value to a specific type. See "Calling Class Methods from Node.js" for details and examples.

```
classMethodBoolean(className, methodName, ...args)   {boolean}
classMethodBytes(className, methodName, ...args)   {Buffer}
classMethodDecimal(className, methodName, ...args)   {Decimal}
classMethodFloat(className, methodName, ...args)   {number}
classMethodInteger(className, methodName, ...args)   {number}
classMethodIRISList(className, methodName, ...args)   {IRISList}
classMethodNumber(className, methodName, ...args)   {number}
classMethodObject(className, methodName, ...args)   {any}
classMethodString(className, methodName, ...args)   {string}
```

*parameters:*

- `className` — fully qualified name of the class to which the called method belongs.
- `methodName` — name of the class method.
- `...args` — zero or more method arguments of supported types. Number, string, and IRISList return values are projected as literals. All other values are projected as proxy objects.

### classMethodVoid()

Iris.**classMethodVoid()** calls an ObjectScript class method with no return value, passing zero or more arguments. See "Calling Class Methods from Node.js" for details and examples.

```
classMethodVoid(className, methodName, ...args)
```

*parameters:*

- `className` — fully qualified name of the class to which the called method belongs.
- `methodName` — name of the class method.
- `...args` — zero or more method arguments of supported types. Number, string, and IRISList return values are projected as literals. All other values are projected as proxy objects.

This method assumes that there will be no return value, but can be used to call any class method. If you use **classMethodVoid()** to call a method that returns a value, the method will be executed but the return value will be ignored. Also see Iris.**classMethodValue()**.

### close()

Iris.**close()** closes the Iris object.

```
close()
```

### get()

Iris.**get()** returns the value of the global node as a type corresponding to the ObjectScript datatype of the property.

This method has a corresponding set of typecast methods (listed below).

```
get(globalName, ...subscripts)   {any}
```

*parameters:*

- `globalName` — global name
- `subscripts` — zero or more subscripts specifying the target node

### Iris.get() Typecast Methods

All of the Iris.**get()** typecast methods listed below work exactly like Iris.**get()**, but also cast the return value to a specific type.

```
getBoolean(globalName, ...subscripts)   {boolean}
getBytes(globalName, ...subscripts)   {ArrayBuffer}
getDecimal(globalName, ...subscripts)   {Decimal}
getFloat(globalName, ...subscripts)   {number}
getInteger(globalName, ...subscripts)   {number}
getList(globalName, ...subscripts)   {IRISList}
getIRISList(globalName, ...subscripts)   {IRISList}
getNumber(globalName, ...subscripts)   {number}
getObject(globalName, ...subscripts)   {any}
getString(globalName, subscripts)   {string}
```

The **getList()** method is retained for backward compatibility. It is identical to **getIRISList()**, which should be preferred.

*parameters:*

- `globalName` — global name
- `subscripts` — zero or more subscripts specifying the target node

### getAPIVersion()

Iris.**getAPIVersion()** returns the version string for this version of the Native SDK. Also see **getServerVersion()**.

```
getAPIVersion()   {string}
```

### getServerVersion()

Iris.**getServerVersion()** returns the version string for the currently connected InterSystems IRIS server. Also see **getAPIVersion()**.

```
getServerVersion()   {string}
```

### getTLevel()

Iris.**getTLevel()** returns the number of nested transactions currently open in the session (1 if the current transaction is not nested, and 0 if there are no transactions open). This is equivalent to fetching the value of the ObjectScript **$TLEVEL** special variable. See "Processing Transactions in Node.js" for details and examples.

```
getTLevel()   {number}
```

### increment()

Iris.**increment()** increments the global node with the *incrementBy* argument and returns the new value of the global node. If there is no existing node at the specified address, a new node is created with the specified value. This method uses a very fast, thread-safe atomic operation to change the value of the node, so the node is never locked. See "Processing Transactions in Node.js" for details and examples.

```
increment(incrementBy, globalName, ...subscripts)   {number}
```

*parameters:*

- `incrementBy` — numeric value to which to set this node (`null` value sets global to `0`, decimal value will be truncated to an integer).
- `globalName` — global name
- `subscripts` — zero or more subscripts specifying the target node

Common usage for $INCREMENT is to increment a counter before adding a new entry to a database. $INCREMENT provides a way to do this very quickly, avoiding the use of the LOCK command. See "$INCREMENT and Transaction Processing" in the *ObjectScript Reference*.

### isDefined()

Iris.**isDefined()** returns a value that indicates whether a node has a value, a child node, or both.

```
isDefined(globalName, ...subscripts)   {number}
```

Returns one of the following integer values:

- `0` if the node does not exist.
- `1` if the node has a value but no child nodes.
- `10` if the node is valueless but has one or more child nodes.
- `11` if the node has a both a value and child nodes.

*parameters:*

- `globalName` — global name
- `subscripts` — zero or more subscripts specifying the target node

### iterator()

Iris.**iterator()** returns an Iterator object for the specified node (see "Class Iterator").

```
iterator(globalName, ...subscripts)   {"intersystems-iris".Iterator}
```

*parameters:*

- `globalName` — global name
- `subscripts` — zero or more subscripts specifying the target node

### kill()

Iris.**kill()** deletes the specified global node and all of its subnodes. If there is no node at the specified address, the command will do nothing. It will not throw an `<UNDEFINED>` exception.

```
kill(globalName, ...subscripts)
```

*parameters:*

- `globalName` — global name
- `subscripts` — zero or more subscripts specifying the target node

### lock()

Iris.**lock()** locks the global. This method performs an incremental lock (you must call the **releaseAllLocks()** method first if you want to unlock all prior locks). Throws a `<TIMEOUT>` exception if the *timeout* value is reached waiting to acquire the lock. See "Concurrency Control with Node.js" for more information.

```
lock(lockType, timeout, lockReference, ...subscript)   {boolean}
```

*parameters:*

- `lockType` — one of the following strings: `"S"` for shared lock, `"E"` for escalating lock, `"SE"` for both, or `""` for neither. An empty string is the default mode (unshared and non-escalating).
- `timeout` — number of seconds to wait to acquire the lock
- `lockReference` — a string starting with a circumflex (^) followed by the global name (for example, `^myGlobal`, *not* just `myGlobal`).

  NOTE: Unlike the `globalName` parameter used by most methods, the `lockReference` parameter *must* be prefixed by a circumflex. Only **lock()** and **unlock()** use `lockReference` instead of `globalName`.
- `subscripts` — zero or more subscripts specifying the target node

See "LOCK" in the ObjectScript Reference for detailed information on locks.

### nextSubscript()

Iris.**nextSubscript()** accepts a node address and returns the subscript of the next sibling node in collation order. This method is similar to $ORDER in ObjectScript.

```
nextSubscript(reversed,globalName, ...subscript)   {null|string}
```

*parameters:*

- `reversed` — boolean true indicates that nodes should be traversed in reverse collation order.
- `globalName` — global name
- `subscripts` — zero or more subscripts specifying the target node

### releaseAllLocks()

Iris.**releaseAllLocks()** releases all locks associated with the session. See "Concurrency Control with Node.js" for more information.

```
releaseAllLocks()
```

### set()

Iris.**set()** assigns *value* as the current node value. The new value may be bool, bytes, bytearray, Decimal, float, int, str, or IRISList.

```
set(value, globalName, ...subscript)
```

*parameters:*

- `value` — new value of the global node
- `globalName` — global name
- `subscripts` — zero or more subscripts specifying the target node

### tCommit()

Iris.**tCommit()** commits the current transaction. See "Processing Transactions in Node.js" for details and examples.

```
tCommit ()
```

### tRollback()

Iris.**tRollback()** rolls back all open transactions in the session. See "Processing Transactions in Node.js" for details and examples.

```
tRollback ()
```

### tRollbackOne()

Iris.**tRollbackOne()** rolls back the current level transaction only. This is intended for nested transactions, when the caller only wants to roll back one level. If this is a nested transaction, any higher-level transactions will not be rolled back. See "Processing Transactions in Node.js" for details and examples.

```
tRollbackOne ()
```

### tStart()

Iris.**tStart()** starts or opens a transaction. See "Processing Transactions in Node.js" for details and examples.

```
tStart ()
```

### unlock()

Iris.**unlock()** decrements the lock count on the specified lock, and unlocks it if the lock count is 0. To remove a shared or escalating lock, you must specify the appropriate *lockMode* (`"S"` for shared, `"E"` for escalating lock). See "Concurrency Control with Node.js" for more information.

```
unlock(lockMode, lockReference, ...subscript)
```

*parameters:*

- `lockMode` — one of the following strings: `"S"` for shared lock, `"E"` for escalating lock, `"SE"` for both, or `""` for neither. An empty string is the default mode (unshared and non-escalating).
- `lockReference` — a string starting with a circumflex (^) followed by the global name (for example, `^myGlobal`, *not* just `myGlobal`).

  NOTE: Unlike the `globalName` parameter used by most methods, the `lock_reference` parameter *must* be prefixed by a circumflex. Only **lock()** and **unlock()** use `lockReference` instead of `globalName`.
- `subscripts` — zero or more subscripts specifying the target node

# 6.4 Class IRISObject

```
FROM V2, 2022-08-24 tutorial-quick_start.htm:

The IRISObject class represents a proxy object created on behalf of IRISdb object. IRISObjects are
created when a function/method returns an IRISdb object, at which point the proxy gets instantiated.
```

Class IRISObject provides methods to work with External Server inverse proxy objects (see "Using Node.js Inverse Proxy Objects" for details and examples).

If the called method returns an object that is a valid OREF, an inverse proxy object (an instance of IRISObject) for the referenced object will be generated and returned. For example, **classMethodObject()** will return a proxy object for an object created by **%New()**. also true of classMethodValue????

## 6.4.1 IRISObject Constructor

The IRISObject constructor takes the following parameters:

```
IRISObject(connection, oref)
```

*parameters:*

- `connection` — a Connection object
- `oref` — the OREF of the database object to be controlled.

## 6.4.2 IRISObject Method Details

### close()

IRISObject.**close()** releases this instance of IRISObject.

```
close()
```

### get()

IRISObject.**get()** fetches a property value of the proxy object.

This method has a corresponding set of typecast methods (listed below).

```
get(propertyName)   {null|any}
```

*parameters:*

- `propertyName` — name of the property to be returned.

### IRISObject.get() Typecast Methods

All of the IRISObject.**get()** typecast methods listed below work exactly like IRISObject.**get()**, but also cast the return value to a specific type. They return null if the property does not exist.

```
getBoolean(propertyName)   {null|boolean}
getBuffer(propertyName)   {null|Buffer}
getDecimal(propertyName)   {null|Decimal}
getFloat(propertyName)   {null|number}
getInteger(propertyName)   {null|number}
getIRISList(propertyName)   {null|IRISList}
getNumber(propertyName)   {null|number}
getObject(propertyName)   {null|any}
getString(propertyName)   {null|string}
```

*parameters:*

- `propertyName` — name of the property to be returned.

### invoke()

IRISObject.**invoke()** invokes an instance method of the object, returning a variable of a type corresponding to the ObjectScript datatype of the property.

This method has a corresponding set of typecast methods (listed below).

```
invoke(methodName, ...arguments)   {any}
```

*parameters:*

- `methodName` — name of the instance method to be called.
- `...arguments` — zero or more arguments of supported types.

Also see **invokeVoid()**, which is used to invoke methods that do not return a value.

### IRISObject.invoke() Typecast Methods

All of the IRISObject.**invoke()** typecast methods listed below work exactly like IRISObject.**invoke()**, but also cast the return value to a specific type.

```
invokeBoolean(methodName, ...arguments)   {boolean}
invokeBytes(methodName, ...arguments)   {Buffer}
invokeDecimal(methodName, ...arguments)   {Decimal}
invokeFloat(methodName, ...arguments)   {number}
invokeInteger(methodName, ...arguments)   {integer
invokeIRISList(methodName, ...arguments)   {IRISList}
invokeNumber(methodName, ...arguments)   {number}
invokeObject(methodName, ...arguments)   {any}
invokeString(methodName, ...arguments)   {string}
```

*parameters:*

- `methodName` — name of the instance method to be called.
- `...arguments` — zero or more arguments of supported types.

### invokeVoid()

IRISObject.**invokeVoid()** invokes an instance method of the object, but does not return a value.

```
invokeVoid(methodName, ...arguments)
```

*parameters:*

- `methodName` — name of the instance method to be called.
- `...arguments` — zero or more arguments of supported types.

### set()

IRISObject.**set()** sets a property of the proxy object.

```
set (propertyName, propertyValue)
```

*parameters:*

- `propertyName` — name of the property to which *value* will be assigned.
- `propertyValue` — new value of the property. Value can be any supported type.

# 6.5 Class Iterator

Class Iterator is a member of external:"intersystems-iris-native". Instances of Iterator are created by calling Iris.**iterator()**. See "Finding Nodes in a Global Array" for more details and examples.

### next()

Iterator.**next()** positions the iterator at the next sibling node in collation order and returns an object containing *done* and *value* properties. If the iterator is at end then *done* is `true`, otherwise it is `false`.

```
next()   {any}
```

When *done* is `false`, the return type of the *value* property can be set by any of the following methods:

- **entries()** causes *value* to return as an array where *value(0)* is the subscript and *value(1)* is the node value (this array is the default when the iterator is created).
- **keys()** causes *value* to return only the current subscript.
- **values()** causes *value* to return only the value of the current node.

### startFrom()

Iterator.**startFrom()** sets the iterator's starting position to the specified subscript. The starting position does not have to be a valid subnode. Returns `this` for chaining.

```
startFrom(subscript)   {external:"intersystems-iris-native".Iterator}
```

*parameter:*

- `subscript` — the subscript to use as a starting point. Does not have to specify an existing node.

The iterator will not point to a node until you call **next()** to advance to the next existing node in collating order.

### reversed()

Iterator.**reversed()** toggles iteration direction between forward and reverse collation order. By default, the iterator is set to forward iteration when it is defined. Returns `this` for chaining.

```
reversed()   {external:"intersystems-iris-native".Iterator}
```

### entries()

Iterator.**entries()** specifies that **next()**.*value* should be an array containing the node subscript (*value(0)*) and node value (*value(1)*). Returns `this` for chaining.

```
entries()   {external:"intersystems-iris-native".Iterator}
```

### keys()

Iterator.**keys()** specifies that **next()**.*value* should contain only the node subscript (key). Returns `this` for chaining.

```
keys()   {external:"intersystems-iris-native".Iterator}
```

### values()

Iterator.**values()** specifies that **next()**.*value* should contain only the node value. Returns `this` for chaining.

```
values()   {external:"intersystems-iris-native".Iterator}
```